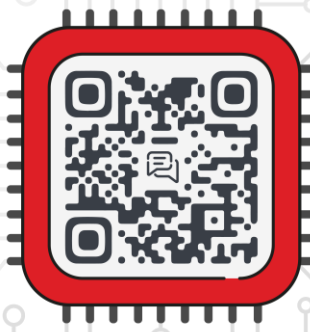




5

8

960 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

Exception

عملي مشترك



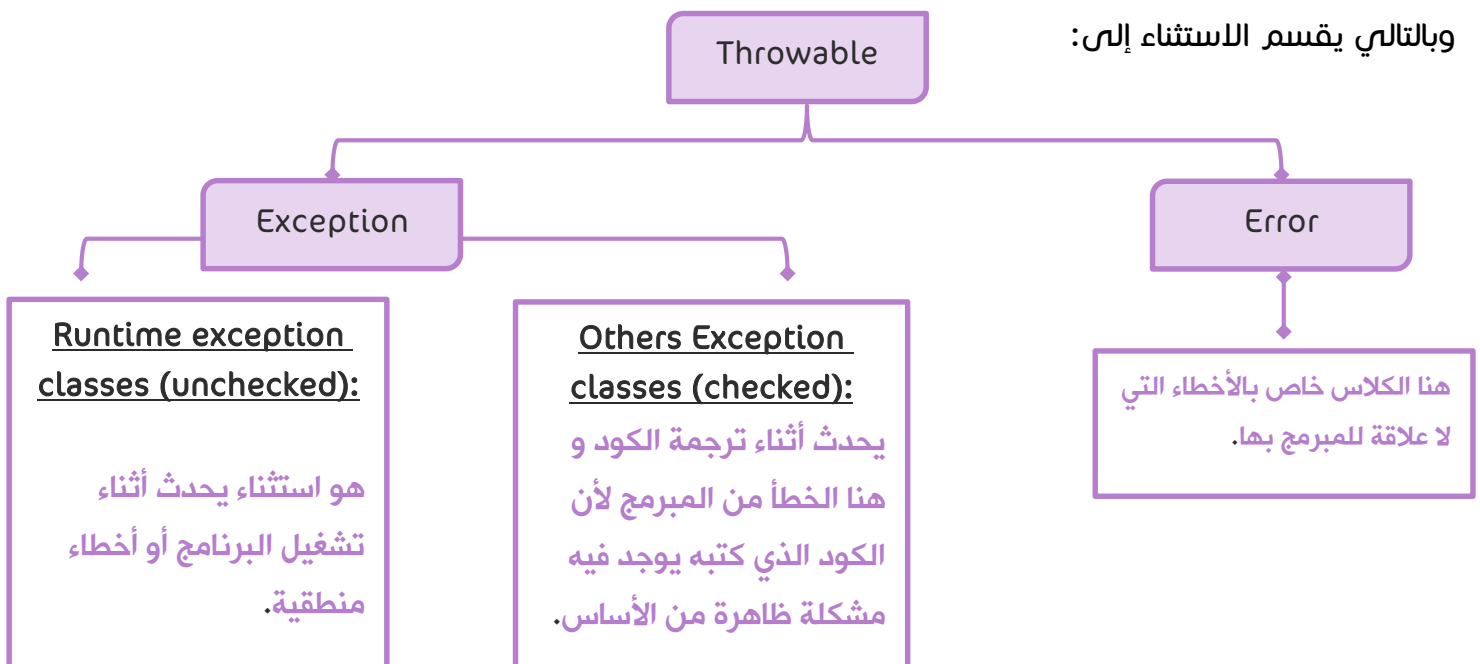
RB Informatics; 20/11/2024

برمجة ٣

في هذه المحاضرة سنبدأ في أول بحث من مادة البرمجة 3 و هو ال "Exception"

تعريفه: الاستثناء بالبرمجة عبارة عن خطأ يحدث أثناء تشغيل البرنامج يؤدي إلى إيقاف البرنامج بشكل غير طبيعي وفي هذه المحاضرة سنتعلم كيفية تجنب حدوث أخطاء في البرنامج الذي نكتبه، وفعلياً سنتعلم كيف نلتقط هذه الأخطاء في حال حدثت حتى لا يتوقف البرنامج مع المستخدم ولا يظهر أي خطأ.

- تم تقسيم الاستثناءات في لغة ال java إلى عدة أنواع وكل نوع تم تمثيله في class منعزل.
- جميع هذه ال classes ترث من class أساسي اسمه **Exception**
- **Exception class** بدوره يرث من class اسمه **Throwable**.
- **إذاً،** أي class يرث من **Exception class** وهو عبارة عن class يمثل استثناء معين.
- الأخطاء التي تسببها الأجهزة والتي تسمى (**error**) تم إنشاء class خاص له اسمه **Error** وهو يرث مباشرة من ال (**Throwable class**).



Exception Keywords:

1. **try**: نضع فيها ال statements التي نتوقع أن يحدث فيها استثناء خلال ال Runtime.
 2. **catch**: نضع فيها التعليمات التي نعالج فيها الخطأ في حال ظهوره.
 3. **finally**: هي أيضا تضمن معالجة الخطأ مثل catch ولكن finally **دوماً تنفذ مهماً كان**
 4. **throw**: تستخدم داخل ال methods للإلقاء Exception بشكل صريح فنكتب throw ومن ثم نوع ال Exception وبما أن كل Exception هو object نسبق النوع ب new لنولد كائن من هذا النوع ونستخدم للإلقاء Exception واحد فقط.
- Throw new Exception Type();**
5. **throws**: تستخدم للتصريح عن ال Exception وتستخدم مع ال methods فتوضع بعد أقواس ال parameters.

ملاحظات:

- نستخدم **try** مع **catch** واحدة على الأقل ولا يصح وضع **try** بدون **catch**.
- نستخدم **try** مع أكثر من **catch** في حال كان داخل ال **try** أكثر من Exception ولكن عندما يدخل ال compiler إلى داخل ال **try** الذي وضعنا فيه عدة استثناءات بمجرد التقاط أول خطأ فلن يكمل قراءة ال **try** وسيدخل مباشرة إلى ال **catch**.
- نستخدم **finally** في حال أردنا تنفيذ أمر معين في كل الأحوال و هي اختيارية.
- **Throw** تستخدم لخلق Exception.
- **Throws** تستخدم لتأجيل معالجة ال Exception

بعض أنواع ال Exception مع ال catch المناسبة لمعالجتها:

ArithmeticException

محاولة القسمة على صفر:

NullPointerException

محاولة الوصول لمتحول null:

StringIndexOutOfBoundsException

محاولة الوصول إلى محرف خارج حدود المتحول string:

FileNotFoundException

قراءة ملف غير موجود:

NumberFormatException

محاولة تحويل (مثلاً من محرف لرقم صحيح):

ArrayIndexOutOfBoundsException

الوصول إلى مكان خارج حدود مصفوفة رقمية:

اكتب برنامج يطلب من المستخدم إدخال أحد الأرقام التالية:

- الرقم واحد من أجل تنفيذ عملية القسمة وعندها نطلب من المستخدم إدخال الرقمين ونعطيه الناتج ويجب مراعاة الـ exception المناسب في حال القسمة على صفر.
- الرقم اثنان من أجل فتح ملف وقراءته وعندها نطلب من المستخدم إدخال اسم الملف ونعرض محتواه في حال وجوده مع مراعاة الـ exception المناسب في حال عدم وجود الملف.
- الرقم ثلاثة يعرض الوزن المثالي للمستخدم عن طريق الطول وعندها نطلب من المستخدم إدخال طوله ونعطيه الوزن المثالي = الطول - 110 ، ويجب بناء exception جديد يعالج حالة الطول الخاطئ مثل الطول 0 أو أكثر من 300 سم مثلاً.

شرح التمرين غير المحلول :



```

1 package exception1;
2 import java.util.Scanner;
3 import java.io.File;
4 import java.io.FileNotFoundException;
5 import java.io.FileReader;
6 import java.io.IOException;
7 public class Exception1 {
8     public static void main(String[] args) {
9         Scanner s=new Scanner(System.in);
10        System.out.println("enter 1:to do
11        division \n enter 2:to read a file \n enter
12        3:to display the perfect length");
13        int o=s.nextInt();
14        switch(o){
15        case 1:{
16            try{
17                double num1;
18                double num2;
19                System.out.println("enter first
20                number:");
21                num1=s.nextDouble();
22                System.out.println("enter second
23                number:");
24                num2=s.nextDouble();
25                double div=num1/num2;
26                System.out.println("div="+div);}

```

❖ أولاً يدخل الـ compiler إلى تابع الـ main وسيدخل لجملة الطباعة في السطر (10) وسيطبع إما أن يدخل المستخدم 1 أو 2 أو 3 .

❖ وبعدها بالسطر (11) سيدخل المستخدم أحد هذه الأرقام وسيدخل إلى switch ولدينا 3 حالات:

1. أدخل المستخدم رقم (1):

وبالتالي سيدخل إلى case 1 بالسطر (13) وبعدها إلى try وسيطلب إدخال رقمين في حال كان الرقمين غير الصفر أكمل وأعطى قيمة القسمة div بالسطر (22) ولن يدخل إلى الـ catch بالسطر (23) أما إذا أدخلنا المقام صفر عندها يدخل للسطر (23) سيظهر الخطأ exception وسيخرج من try ويدخل لـ catch ويطبع العبارة بالسطر (24).

2. أدخل المستخدم رقم (2):

عندها سيدخل إلى case 2 بالسطر (26) وبما أن المطلوب إدخال اسم الملف وعرض محتواه في حال وجوده وبالتالي يدخل الـ compiler للسطر (28) يطلب إدخال اسم الملف وبعد أن يدخله المستخدم وبعد إنشاء الـ object الملف file و لكي تستطيع عرض محتوى الملف استخدمنا حلقة while لتقرأ جميع أحرف النص داخل الملف في كل دورة تقرأ حرف من الملف إذا وجدته تطبعه ، والمتحول d هو عداد يأخذ قيمة الحرف الذي تريد قراءته (قيمة مكانه) ونطبعه بالسطر (34) بعد أن قمنا بـ casting للمتغير d ليعرض لنا الحرف وليس قيمة مكانه في حال وجود الملف

✓ وفي النهاية نقوم بإغلاق الكائن المتصل بالملف [وكما ذكرنا أن الملف يجب إغلاقه بتابع close() كما في السطر (35)]

```

23 catch(ArithmeticException e){
24     System.out.println("we can't divide a
25     number by zero");}
26 break;}
27 case 2:{
28     try {
29         System.out.println("enter the name of the
30         file that you want to open it:");
31         String name=s.next();
32         File file=new File(name);
33         FileReader r = new FileReader(file);
34         int d = 0;
35         while ((d = r.read())!=-1)
36             {System.out.println((char)d);}
37         r.close();
38         catch (FileNotFoundException ex) {
39             System.out.println("file not found");}
40         catch(IOException e){
41             System.out.println("input | output
42             exception");}
43         break;}
44     case 3:{
45         try{
46             System.out.println("enter the length");
47             int length=s.nextInt();
48             if(length<300&&length>0){
49                 double p_length=length-110;
50                 System.out.println("p_length="
51                 +p_length);}
52             else{
53                 throw new p_exception ("tall is not
54                 true");}}
55         catch(p_exception e){
56             System.out.println("tall is not true");}
57         break;}
58         default:{
59             System.out.println("please enter another
60             number ");}}}}

```

✓ أما في حال عدم وجوده يخرج من try إلى ال catch بالسطر (36) ويطبع عبارة الطباعة بالسطر (37) <لا يوجد ملف> ونضع: ✓

Catch (IO Exception e)
والتي تطبع input /output Exception

ملاحظة:

في حال عدم وضع catch سيظهر error بالسطر الخاص بال read() والسطر الخاص ب close() .

3. في حال إدخال رقم (3):

سيدخل لل case3: بالسطر (41) وبعدها ل try وبعدها سيطبع للمستخدم أن يدخل ال length في حال كان: إما:

$Length < 300$,, $length > 0$

وعندها سيكون الوزن المثالي (p_weight) يساوي: $p_weight = length - 110$ وبعدها سيطبع الوزن.

أو:

$Length < 300$,, $length < 0$

كما طلب منا بنص السؤال (بناء exception جديد يعالج حالة الطول الخاطئ) ولذلك أنشأنا (1) File صف ال P_exception يرث من ال exception ومررنا $.tall$ is not true .

```

1 package exception1;
2 public class p_exception extends
3     Exception {
4     String tallisnottrue;
5     public p_exception(String
6         tallisnottrue){
7         this.tallisnottrue = tallisnottrue;}
8     @Override
9     public String toString()
10         return tallisnottrue;}

```

التمارين المحلولة:

التمرين الأول:

عرف الصف LabFirstExceptions وفيه التابع الرئيسي main الذي يقوم بالعمليات التالية:

1. محاولة القسمة على صفر مع معالجة ال exception المناسب وطباعة العبارة المناسبة.
2. محاولة الوصول إلى متحول null مع معالجة ال exception المناسب وطباعة العبارة المناسبة.
3. محاولة الوصول إلى محرف خارج حدود المتحول string مع معالجة ال exception المناسب وطباعة العبارة المناسبة.

```

1 package exception2;
2 public class labFirstExceptions {
3     public static void main(String[] args){
4         try{
5             int a=30,b=0;
6             int d=a/b;
7             System.out.println("result="+d);}
8         catch(ArithmeticException e){
9             System.out.println("cannot divide a number
10 by zero");}
11         try{
12             String x=null;
13             System.out.println(x.charAt(0));}
14         catch(NullPointerException e){
15             System.out.println("Null Pointer
16 Exception");}
17         try{
18             String a="this is practical session";
19             char c=a.charAt(30); }
20         catch(StringIndexOutOfBoundsException
21 e){
22             System.out.println("String index out of
23 bounds exception");
24     }}}

```

شرح التمرين 1:

- ❖ يدخل ال compiler إلى تابع ال main ويدخل إلى try في السطر (4)
- ❖ عندما يصل للسطر (6) عندها يلتقط الخطأ (exception) وهو القسمة على صفر
- ❖ ثم يخرج من try ويدخل ال catch بالسطر (8) ويطبع العبارة.
- ❖ ثم يدخل إل try بالسطر (10) ، عرفنا ضمنه المتحول من نوع string
- ❖ بالسطر (17) محاولة الوصول إلى محرف خارج حدود المتحول (a)
- ❖ عندها سيخرج من try ويدخل إلى catch بالسطر (18) و يطبع العبارة المناسبة ومن ثم ينهي البرنامج.

لو كتبنا بالسطر (17)



```
char c=a.charAt(25);
```

فلن يدخل إلى catch في السطر (21) لأن العدد 25 ضمن حدود عدد محارف المتحول a .

التمرين الثاني:

عرف الصف LabSecondException وفيه التابع الرئيسي *main* الذي يقوم بالعمليات التالية:

- 1- محاولة قراءة ملف غير موجود .
- 2- محاولة تحويل محارف إلى رقم صحيح.
- 3- محاولة الوصول إلى مكان خارج حدود مصفوفة رقمية، مع معالجة جميع ال Exception السابقة ضمن try واحدة وطباعة عبارة مناسبة عند الانتهاء من البرنامج سواء انتهى التنفيذ بشكل صحيح أو حصل Exception باستخدام finally.

```

1 package labsecondexception;
2 import java.io.File;
3 import java.io.FileNotFoundException;
4 import java.io.FileReader;
5 public class LabSecondException {
6     public static void main(String[] args) {
7         try{
8             File f=new File("E://fill.txt");
9             FileReader fr=new FileReader(f);
10            int sum=Integer.parseInt("akki");
11            System.out.println(sum);
12            int a[]=new int[5];
13            a[6]=9;}
14            catch(FileNotFoundException e){
15                System.out.println("fill dose not exist");}
16            catch(NumberFormatException |
17                ArrayIndexOutOfBoundsException e){
18                System.out.println("number format
19                exception");
20                System.out.println("array index out of
21                exception");}
22            finally{System.out.println("finally code
23                always excute");}}

```

شرح التمرين 2:

- ❖ السطر (2 و 3 و 4) قمنا بتدخيل المكتبات
- ❖ ثم يدخل إلى تابع ال main ثم try بالسطر (7) والتي تتضمن 3 أنواع من ال exception .
- 1. محاولة قراءة ملف غير موجود
- 2. محاولة تحويل محارف إلى رقم صحيح
- 3. محاولة الوصول إلى مكان خارج حدود مصفوفة رقمية
- ❖ ولكن ال compiler بمجرد أن يلتقط أول exception وهو هنا (محاولة قراءة ملف غير موجود) فلن يكمل دخول try وسيخرج منها إلى catch المناسبة بالسطر (14) وسيطبع عبارتها.
- ❖ وبالتالي لن يدخل catch بالسطر (16) التي تتضمن معالجة النوعين الآخرين من ال exception وسيدخل فوراً إلى finally بالسطر (19) وسيطبع عبارتها.

OUTPUT

```

File does not exist
Finally code always
excute

```

- + يمكن معالجة أكثر من exception داخل catch واحدة ولكن يجب الفصل بينها ب (....|....) كما في السطر (16)
- + كما ذكرنا سابقاً عندما نضع finally ضمن الكود عندها إجبارياً ال compiler سيدخل إليها

التمرين الثالث:

عرف exception جديد اسمه `insufficientFundsException` والذي يحوي الوصفة `amount` مع الباني المناسب والباني `getAmount` ،

ثم عرف صف `checkingAccount` وفيه ال Attribute `balance` وال `withdraw` method التي تأخذ الكمية `amount` كدخل وتطرحه من الرصيد `balance` ولكنها ترمي ال `insufficientFundsException` في حال كانت الكمية المطلوب سحبها أكبر من الرصيد المتبقي.

ثم عرف الصف الرئيسي الذي يحوي التابع الرئيسي `main` ويقوم باختبار عمليات السحب والايذاع على قيم محددة ومعالجة وطباعة ال exception في حال حدوثه.

```
1 package insufficientFundsException;
2 import java.io.*;
3 public class insufficientFundsException extends Exception{
4     private double amount;
5     public insufficientFundsException (double amount) {
6         this.amount = amount;
7     }
8     public double getAmount() {
9         return amount;
10    }
11    class CheckingAccount{
12        private double balance;
13        public CheckingAccount(double balance) {
14            this.balance = balance;
15        }
16        public void deposit(double amount){
17            balance+=amount;
18        }
19        public void withdraw(double amount)
20            throws insufficientFundsException {
21            if(balance>=amount){
22                balance-=amount;
23            }
24            else{
25                double needs =amount-balance;
26                throw new insufficientFundsException(needs);
27            }
28        }
29        public double getBalance() {
30            return balance;
31        }
32    }
33 }
```

```
1 package insufficientFundsException;
2 public class main{
3     public static void main(String[] args) {
4         CheckingAccount c=new CheckingAccount(101);
5         System.out.println("Depositing 500$");
6         c.deposit(500.00);
7         try{
8             System.out.println("widthdrawing 50");
9             c.withdraw(500.00);
10            System.out.println("widthdrawing 100$");
11            c.withdraw(100.00);
12            System.out.println("widthdrawing 600$");
13            c.withdraw(600.00);
14        } catch (insufficientFundsException e){
15            System.out.println("sorry but u are short$");
16            +e.getAmount();
17            e.printStackTrace();
18        }
19    }
20 }
```

لا تستسلم أبداً ، الأشياء العظيمة تحتاج إلى وقت..

تذكرة:

لقد ذكرنا أن الدالتين:

`getAmount()``PrintStackTrace()`

تستخدم لإرجاع رسالة بسيطة تظهر لنا فيها الخطأ الذي حدث.

**شرح التمرين 3 :**

- ❖ السطر (3) تعريف الصف العام Exception
- ❖ السطر (15,21) تعريف التابع `withdraw` الذي يرمي Exception الجديد `insufficientFundsException` في حال كانت الكمية المسحوبة أكبر من الرصيد.
- ❖ ثم دخلنا إلى تابع الـ `main` عرفنا `object` من الصف `CheckingAccount` برصيد (101) في السطر (4)
- ❖ بالسطر (6) نقوم بإيداع 500 ثم ندخل الـ `try` بالسطر (7) ثم نسحب 500 ثم 100 ثم 600 وهنا سوف يسبب حدوث الـ `exception` الجديد لأننا تجاوزنا الرصيد
- ❖ ثم يدخل الـ `catch` في السطر (14).
- ❖ ومن ثم يقوم بطباعة رسالة `error` في حال حدوث الـ `exception`

⬅ لعدة عن الـ java IO :

(Files IO _ java IO) : IO تعني (input - output).

1. `File f=new File();` لإنشاء file
2. `FileReader r=new FileReader();` لقراءة file.
3. `FileWriter w=new FileWriter();` لكتابة file

كل دالة نقوم باستعمالها لا ننسى إغلاقها بتعليمة `[close();]`مثال: `read()` , `write()`نضع `IO Exception` لأنالدالة `close()` التي تقوم باستدعائهابعد كل دالة قد ترمي `exception`في تابع الـ `main()`.

The End